

1. Objetivos

Ao final desta prática o estudante deverá ser capaz de:

- Representar grafos utilizando listas de adjacência;
 - Implementar busca em profundidade (DFS);
 - Implementar busca em largura (BFS);
 - Compreender e implementar ordenação topológica;
 - Identificar árvores geradoras;
 - Construir árvores geradoras mínimas utilizando Prim;
 - Implementar o algoritmo de Dijkstra;
 - Comparar complexidades computacionais dos algoritmos estudados.
-

2. Fundamentos Teóricos

2.1 Grafos

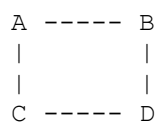
Um grafo é definido por:

$$\begin{bmatrix} G = (V, E) \end{bmatrix}$$

onde:

- V = conjunto de vértices
- E = conjunto de arestas

Exemplo:



2.2 Lista de Adjacência

Representação eficiente para grafos esparsos.

Exemplo:

```
0 -> 1 -> 2
1 -> 0 -> 3
2 -> 0 -> 3
```

3 -> 1 -> 2

Estrutura:

```
typedef struct No{
    int destino;
    struct No *prox;
}No;

typedef struct{
    int V;
    No **lista;
}Grafo;
```

Complexidade espacial: $O(V+E)$

2.3 Busca em Profundidade (DFS)

Explora um caminho completamente antes de retroceder.

Exemplo:

```
A -> B -> D
|
C
```

Percurso:

A B D C

Complexidade: $O(V+E)$

2.4 Busca em Largura (BFS)

Visita primeiro todos os vizinhos. Utiliza fila.

Exemplo:

```
A
| \
B  C
|
D
```

Percurso:

A B C D

Complexidade: $O(V+E)$

2.5 Ordenação Topológica

Aplicável somente a DAG (*Directed Acyclic Graph*). Produz uma ordem válida de execução.

Exemplo de grafo:

```
Compilar (nó)
  |
Testar (nó)
  |
Executar (nó)
```

Resultado:

Compilar -> Testar -> Executar

Complexidade: $O(V+E)$

2.6 Árvore Geradora

Uma árvore geradora:

- conecta todos os vértices;
- não possui ciclos;
- possui exatamente $V-1$ arestas.

2.7 Árvore Geradora Mínima (AGM)

Utilizada em:

- redes de computadores;
- redes elétricas;
- roteamento.

Nesta prática será utilizado o algoritmo de Prim.

Complexidade: $O(V^2)$ ou $O((V+E)\log V)$ dependendo da implementação.

2.8 Menor Caminho – Dijkstra

Determina o menor custo entre um vértice origem e todos os demais.

Requer pesos não negativos.

Complexidade: $O(V^2)$ na versão básica.

3. Código Base

```
typedef struct No{
    int destino;
    int peso;
    struct No *prox;
}No;

typedef struct{
    int V;
    No **lista;
}Grafo;
```

Atividade 1 – Construção do Grafo

Complete a função

```
void adicionarAresta(Grafo *g,
                    int origem,
                    int destino,
                    int peso){

    No *novo = _____;

    novo->destino = destino;
    novo->peso = peso;

    novo->prox = _____;
    _____;

}
```

Perguntas

1. Qual a complexidade da inserção?
2. Qual a vantagem da lista de adjacência sobre matriz de adjacência?
3. Quando a matriz de adjacência pode ser mais vantajosa?

Atividade 2 – Busca em Profundidade (DFS)

Complete a função

```
void DFS(Grafo *g,
        int v,
        int visitado[]){

    visitado[v] = 1;

    printf("%d ", v);

    No *aux =
        _____;

    while(aux != NULL){
```

```

        if(!visitado[
            _____ ] ){
            _____;
        }
        aux = aux->prox;
    }
}

```

Perguntas

1. Qual estrutura implícita é utilizada pela DFS?
2. A DFS encontra sempre o menor caminho?
3. Cite duas aplicações da DFS.

Atividade 3 – Busca em Largura (BFS)

Complete a função

```

void BFS(Grafo *g,int origem){

    int fila[100];
    int inicio = 0;
    int fim = 0;

    int visitado[100]={0};

    fila[fim++] = origem;

    visitado[origem] = 1;

    while(inicio < fim){

        int v =
            _____;

        printf("%d ",v);

        No *aux =
            _____;

        while(aux){

            if(!visitado[
                aux->destino ]){

                visitado[
                    aux->destino ] = 1;

                _____;

            }

            aux = aux->prox;

        }

    }
}

```

```
}
```

Perguntas

1. Qual estrutura é essencial para BFS?
2. BFS encontra o menor caminho em grafos sem peso?
3. Compare BFS e DFS.

Atividade 4 – Ordenação Topológica

Complete

```
void topoDFS(Grafo *g,int v,int visitado[],int pilha[],int *topo){  
    visitado[v] = 1;  
  
    No *aux = g->lista[v];  
  
    while(aux){  
        if(!visitado[_____]){  
            _____;  
        }  
        aux = aux->prox;  
    }  
    pilha[(*topo)++] = _____;  
}
```

Perguntas

1. Em quais grafos a ordenação topológica pode ser aplicada?
2. O que acontece se existir ciclo?
3. Cite uma aplicação prática.

Atividade 5 – Árvore Geradora

Considere o grafo:

```
0---1  
|  / |  
|  / |  
2---3
```

Perguntas

1. Quantas arestas possui uma árvore geradora?
2. Apresente uma árvore geradora válida.
3. O conjunto abaixo é uma árvore geradora?

(0,1)
(1,3)
(3,2)

Justifique.

Atividade 6 – Prim (Árvore Geradora Mínima)

Complete

```
for(i=0;i<V-1;i++){  
    int u = _____;  
    visitado[u] = 1;  
    No *aux = g->lista[u];  
    while(aux){  
        int v = aux->destino;  
        if(!visitado[v] &&  
            aux->peso <  
            chave[v]){  
            _____;  
            _____;  
        }  
        aux = aux->prox;  
    }  
}
```

Perguntas

1. Qual o objetivo de Prim?
2. Prim funciona em grafos desconexos?
3. Diferença entre árvore geradora e AGM.

Atividade 7 – Dijkstra

Complete

```
for(i=0;i<V;i++){  
    int u = _____;  
    visitado[u] = 1;  
    No *aux = g->lista[u];  
    while(aux){
```

```

        int v = aux->destino;

        if (dist[u] + aux->peso
            < dist[v]) {
            _____;
        }

        aux = aux->prox;
    }
}

```

Perguntas

1. Qual restrição existe para os pesos?
2. Qual vetor armazena os menores custos?
3. Dijkstra produz uma árvore?

Atividade 8 – Análise de Execução (Passo a Passo)

Considere:

```

0 --2-- 1
|       /|
4      1 |
|  /    |
2 --3-- 3

```

Partindo do vértice 0:

Execute manualmente o algoritmo de Dijkstra.

Preencha:

Iteração Vértice escolhido Distâncias

Inicial

1

2

3

Final

Perguntas

1. Qual o menor caminho de 0 para 3?
 2. Qual o custo total?
 3. Qual seria o resultado de uma BFS a partir de 0?
-

Atividade 9 – Complexidade

Complete:

Algoritmo Complexidade

DFS _____

BFS _____

Topológica _____

Prim _____

Dijkstra _____

Atividade 10 – Questão Integrada

Considere um sistema de navegação urbana.

Explique:

1. Onde utilizar DFS?
2. Onde utilizar BFS?
3. Onde utilizar Prim?
4. Onde utilizar Dijkstra?
5. Qual algoritmo seria utilizado para definir a ordem de execução de tarefas dependentes?